

Microsoft Azure SQL migration guidance based on the latest supported platform direction and current tooling ecosystem.

1. Migration to Azure SQL Database (2026 Current Guidance)

Azure SQL Database

Positioning (Current)

Azure SQL Database is the **default cloud-native relational database PaaS** for:

- New application development
- Microservices
- SaaS workloads
- Independent database architectures

Supported Migration Approach

Primary Microsoft-supported paths:

- Azure Database Migration Service (online/offline)
- Backup-based migration (limited scenarios)
- Data sync via ETL pipelines (ADF / Fabric)
- Application-led migration (re-architected workloads)

Current Capabilities

Fully managed platform:

- Automatic patching
- Automated backups
- Built-in HA (no configuration required)
- Zone redundancy (service tier dependent)
- Auto-scaling (serverless + elastic pools)
- Hyperscale architecture for large databases

Key Constraints

1. Database-scoped architecture

- No instance-level scope
- No cross-database joins without external patterns

2. Unsupported SQL Server features

- SQL Server Agent (replaced by external orchestration)
- FILESTREAM
- xp_cmdshell
- CLR (limited / restricted scenarios)
- Service Broker (limited)
- Linked Servers (not supported in traditional form)

3. Authentication model (Updated)

- Microsoft Entra ID authentication is primary
- SQL authentication still supported
- No classic Windows-integrated authentication model like on-prem SQL Server

Migration Reality (2026)

Azure SQL Database is best when:

- You can **refactor dependencies**
- You can eliminate **cross-database coupling**
- You want **minimal infrastructure management**
- You are building **cloud-native systems**

2. Migration to Azure SQL Managed Instance (MI)

Azure SQL Managed Instance

Positioning (Current 2026)

Azure SQL Managed Instance is still:

Microsoft's PRIMARY enterprise lift-and-shift target for SQL Server modernization

It is designed for:

- Minimal application change migration
- Instance-level compatibility
- Enterprise SQL Server workloads

Supported Migration Methods (Current)

Fully supported Microsoft methods:

- Native backup/restore (BLOB-based)
- Log Replay Service (LRS)
- Managed Instance Link (Always On-based replication)
- Azure Database Migration Service (DMS)
- Azure Arc-assisted migration workflows

Compatibility Status (2026 Reality)

High compatibility areas:

- SQL Server Agent (fully supported)
- Cross-database queries
- Linked Servers (partial but functional)
- SQL CLR (with restrictions)
- Service Broker
- Database Mail
- SQL Server instance-level objects

Service Tiers (Important Update)

General Purpose

- Remote storage architecture
- Higher IO latency than VM local disk
- Suitable for general workloads

Business Critical (Important correction)

- Local SSD storage
- Lowest latency PaaS SQL option

- Supports:
 - In-Memory OLTP
 - Read replicas
 - Higher throughput workloads

Current Key Limitations

Still NOT supported:

- Full OS access
- FILESTREAM (requires redesign or VM)
- Direct disk/file system access
- Custom Windows services or executables
- User-managed failover clustering (fully platform managed)

Modern Migration Reality (2026)

Managed Instance is the best choice when:

- You need **SQL Agent**
- You rely on **cross-database dependencies**
- You want **minimal rewrite migration**
- You need **enterprise PaaS without VM management**

3. Migration to SQL Server on Azure VM

SQL Server on Azure Virtual Machines

Positioning (2026 Current)

Azure VM SQL is:

The **ONLY** option that provides full SQL Server feature parity in Azure

It is still the default choice for:

- Legacy applications
- Vendor-certified workloads
- Highly customized SQL Server environments
- IO-sensitive OLTP systems requiring full control

Supported Migration Methods

- Native backup/restore
- Log shipping
- Always On Availability Groups
- Replication
- Storage-level migration (disk attach/detach patterns)
- Azure Site Recovery (VM-level)

Full Feature Support (Unique Advantage)

Only Azure VM supports ALL of the following simultaneously:

- Full SQL Server Agent
- FILESTREAM
- xp_cmdshell
- SSIS runtime (native)
- SSRS server hosting

- Windows Authentication (full AD)
- Custom DLLs / OS integrations
- Third-party monitoring agents
- Full control over TempDB layout
- Disk striping / IO tuning

Trade-off (Still unchanged in 2026)

You manage everything:

- Patching
- HA configuration
- DR architecture
- Backup strategy
- Security hardening
- Scaling strategy

Why organizations still choose VM in 2026

- Predictable IO performance
- Vendor constraints
- Legacy dependency constraints
- Complex SSIS / ETL pipelines
- Strict regulatory control over OS layer

Final 2026 Summary (Correct Decision Logic)

Azure SQL Database

Best for:

- Cloud-native applications
- Microservices
- Independent databases
- Low operational overhead

Azure SQL Managed Instance

Best for:

- Enterprise SQL Server migration
- Minimal code changes
- SQL Agent dependency
- Cross-database workloads
- Most “lift-and-shift” scenarios

SQL Server on Azure VM

Best for:

- Full SQL Server compatibility requirement
- Legacy/vendor applications
- OS-level dependencies
- High-performance tuned workloads
- SSIS/SSRS-heavy systems

Key 2026 Reality (Important)

Microsoft's current strategy is:

- Azure SQL Database → cloud-native future
- Azure SQL Managed Instance → enterprise migration path
- SQL Server on Azure VM → compatibility fallback + legacy bridge

1. Feature Parity Matrix (2026 – Current State)**Legend**

- ✓ = Fully supported
- ● = Partially supported / constrained
- ✕ = Not supported

Core Engine & Instance Features

Capability	Azure SQL Database	Azure SQL Managed Instance	SQL Server on Azure VM
Full SQL Server engine parity	✕	●	✓
Instance-level objects	✕	✓	✓
SQL Server Agent	✕ (no native Agent)	✓	✓
Cross-database queries	✕	✓	✓
Linked Servers	✕	●	✓
CLR integration	● (restricted)	●	✓
Service Broker	✕ / limited	✓	✓
DB Mail	✕	✓	✓
FILESTREAM	✕	✕	✓
xp_cmdshell	✕	✕	✓

Security & Authentication

Capability	Azure SQL Database	Azure SQL Managed Instance	SQL Server on Azure VM
SQL Authentication	✓	✓	✓
Microsoft Entra ID	✓	✓	✓
Windows Authentication (AD)	✕	● (hybrid Kerberos via Entra)	✓
Kerberos/NTLM directly	✕	●	✓

HA / DR Capabilities

Capability	Azure SQL Database	Azure SQL Managed Instance	SQL Server on Azure VM
Built-in HA	✓	✓	✗
Auto failover	✓	✓	🕒 (user configured)
Geo-replication	✓	✓	✓ (manual setup)
Failover groups	✓	✓	✗
Always On AG	✗	✗	✓
FCI support	✗	✗	✓

Performance Architecture

Capability	Azure SQL Database	Azure SQL Managed Instance	SQL Server on Azure VM
Dedicated local SSD option	Business Critical only	Business Critical only	✓
Full IO control	✗	✗	✓
TempDB tuning control	✗	✗	✓
Horizontal scale-out	✓ (Hyperscale)	Limited	Manual
Read replicas	✓	✓ (tier dependent)	Manual

Best Fit Summary

- Azure SQL Database → cloud-native / refactored apps
- Managed Instance → enterprise lift-and-shift
- Azure VM → full compatibility / legacy systems

2. Enterprise Migration Decision Tree (2026)

Use this structured decision flow:

Step 1 — Do you require full SQL Server engine behavior?**YES →**

Go to Azure VM

Because you need:

- FILESTREAM
- xp_cmdshell
- FCI / AG control
- OS-level dependencies
- SSIS/SSRS full control
- Third-party agents

NO →

Go to Step 2

Step 2 — Do you depend on any of the following?

- SQL Server Agent jobs
- Cross-database queries
- Linked Servers
- Service Broker
- Database Mail
- Instance-level objects

YES →

Go to Azure SQL Managed Instance

NO →

Go to Step 3

Step 3 — Can you refactor application to be database-scoped?**YES →**

Use Azure SQL Database

Best for:

- SaaS apps
- Microservices
- Modern APIs
- Independent schemas

NO →

Fallback:

- Managed Instance (preferred)
- Azure VM (if tight coupling exists)

Step 4 — Performance / IO constraint check

If workload is:

- Ultra low latency OLTP
- Heavy TempDB usage
- Storage-sensitive workloads

Then:

- Prefer Azure VM OR Managed Instance Business Critical

Step 5 — Data size consideration

Size	Recommendation
< 100 GB	Azure SQL DB or MI
100 GB – multi TB	Managed Instance / Hyperscale
Very large + IO heavy	Azure VM

3. “What Breaks During Migration” Checklist (Critical Reality Layer)

This is what actually fails in real enterprise migrations.

A. Azure SQL Database — Breaking Points

Application breaks if you have:

- Cross-database joins
- SQL Agent job dependency
- File system dependency
- CLR external access
- Linked server usage
- SSIS running inside SQL Server
- Legacy T-SQL relying on system tables
- TempDB tuning assumptions

Architectural rewrite required for:

- Monolithic databases
- Shared DB schemas across apps
- Tight coupling between DBs
- File-based ingestion pipelines

B. Azure SQL Managed Instance — Breaking Points

Still problematic:

1. FILESTREAM workloads

- Must be redesigned or moved to blob storage

2. OS-level dependencies

- Custom Windows services
- File system polling jobs
- Local disk workflows

3. Some performance-sensitive workloads

- Heavy IO with General Purpose tier
- TempDB-intensive workloads (if mis-tiered)

4. SQL Server edge features not fully portable

- Some CLR assemblies
- Some replication edge cases
- Some linked server authentication patterns

Migration friction points:

- Instance collation mismatches
- Login SID mapping issues
- Agent job dependency chains
- SSIS package external dependencies

C. SQL Server on Azure VM — Breaking Points

VM has **no feature breaks**, but operational breaks include:

1. Operational complexity

- HA must be built (AG/FCI)
- Backups must be designed
- Monitoring must be implemented

2. Cloud transformation gap

- No PaaS automation benefits
- Manual patching lifecycle
- Capacity planning required

3. Misconfiguration risks

- Storage misalignment (IOPS issues)
- TempDB misconfiguration
- Improper AG setup
- Backup retention failure

Final 2026 Strategic Summary

Azure SQL Database

Best when:

- You can modernize application design
- You want lowest operational overhead
- You do not need instance-level SQL Server features

Azure SQL Managed Instance

Best when:

- You are migrating enterprise SQL Server as-is
- You need SQL Agent and cross-db support
- You want PaaS without rewriting workloads

SQL Server on Azure VM

Best when:

- You need full SQL Server fidelity
- You have vendor or legacy constraints
- You require OS-level control and predictability